

AgriAid: Digital Community Resource & Farm Management System

Project Proposal for UCS503

Submitted to: Ms. Anushka

Thapar Institute of Engineering and Technology

Anisa Arora, 1024030285
Neha Bansal, 1024030306

Lovish Bansal, 1024030297
Vaibhav Budhia, 1024030307

August 19, 2026

1 Higher-order goal

This project aims to improve the operational efficiency, financial transparency, and accessibility of government welfare for Indian farmers by digitizing non-agronomic operational workflows around the farming ecosystem.

While the broader objective is to empower rural agricultural communities through shared economics and streamlined record-keeping, the immediate engineering goal is to translate informal, offline farm coordination workflows into a measurable, secure, and deployable software platform.

The proposed system, **AgriAid**, connects farmer identity, available agricultural machinery and repair services, local short-term labor requests, farm financial expense/revenue ledgers, rule-based government scheme eligibility matching, and agricultural insurance claim document organization within a single role-based platform.

The project explicitly focuses on **operational coordination and workflow engineering** rather than agronomic advice or clinical crop diagnosis. Decision-making remains entirely under the control of the farmer; the system serves strictly as an information organizer and matching facilitator.

2 Time-to-value

- We will deliver meaningful increments quickly by implementing the smallest complete operational workflow first: user authentication → resource registration → resource discovery → basic booking request → farm ledger entry.
- We will prioritize a working end-to-end operational workflow over attempting to implement all three umbrella modules simultaneously.
- We will initially implement basic manual location input and simple resource matching so that the core booking and financial recording workflow can be validated independently of advanced geo-fencing or mapping integration.
- Location-based mapping (Leaflet / OpenStreetMap integration) will be introduced after experimentally validating coordinate handling and distance matching logic across web browsers.
- We will use short development iterations and continuous testing so that integration issues in core modules are identified early.

- CI/CD pipelines will be introduced early to automatically build, test, and deploy changes to a staging environment (Vercel / Render / Railway).
- Success in the first iteration will be defined by whether a farmer can successfully list a resource, book a slot, and log an associated financial entry rather than by the sheer number of features implemented.

3 Problem Statement

Indian farmers possess extensive practical knowledge of farming and crop production. The difficulties they face most acutely are not agronomic but operational—they arise around the farming ecosystem rather than within the act of farming itself. Current manual processes create several operational bottlenecks:

- **Inaccessible Machinery & Services:** Farmers struggle to access expensive machinery (tractors, harvesters, rotavators) and local service providers on demand during critical seasonal windows.
- **Inefficient Resource Sharing:** Resource sharing and transport pooling to mandis (produce markets) are arranged informally through word-of-mouth, leading to high individual costs and idle equipment capacity.
- **Fragmented Expense Tracking:** Farm income, labor costs, and operational expenses are rarely recorded in a structured digital format, making seasonal financial analysis difficult.
- **Opaque Government Benefits & Claims:** Understanding eligibility rules for government schemes and assembling required documentation for insurance claims is complex, leading to missed financial aid.
- **Lack of Centralized Records:** Farmer profiles, machinery listings, labor requests, financial records, and scheme applications exist in disconnected or manual paper formats.

Why this matters now (contextual relevance): Rural farming communities operate on tight seasonal schedules with limited financial margins. Digitizing operational coordination, transport pooling, and eligibility matching reduces avoidable expenses and administrative delays, significantly improving farm sustainability.

4 Proposed Solution

4.1 Overview

Build a web-based platform that digitizes rural farm coordination while maintaining distinct roles and responsibilities for Farmers, Resource Providers, and System Administrators.

The proposed AgriAid platform integrates three umbrella systems:

1. **Community Resource Network:** Enables discovery, individual/group booking, and transport pooling for agricultural machinery, alongside an equipment breakdown network for emergency repairs.
2. **Farm Operations & Financial Management:** Coordinates local short-term labor requests and maintains structured expense/revenue ledgers by crop and season.
3. **Government Benefits & Claims Assistance:** Matches farmer profiles against scheme eligibility rules and organizes required documentation for agricultural insurance claims.

4.2 Core workflow

1. A farmer authenticates into AgriAid using secure JWT-based credentials and maintains a profile (land size, region, crop type).
2. The farmer searches for nearby available machinery or post a local labor requirement.
3. For machinery/services, the farmer submits an individual or group booking request.
4. The resource provider receives notification of the request and accepts, rejects, or reschedules it.
5. The system checks for scheduling overlaps and updates the reservation state.
6. Following farm operations or produce delivery, the farmer logs transaction details into the Farm Ledger (expense or sales revenue).
7. To access state benefits, the system runs a rule-based matching engine against the farmer's profile to list eligible government schemes and application checklists.
8. In case of crop damage, the farmer uploads photos and incident data to create an organized insurance claim folder.

4.3 Operational constraints for an educational campus setting

- **Academic Timeline:** The system must be designed, implemented, and evaluated within a single 12-week academic semester by a 3–4 member student engineering team.
- **Usability for Low Digital Literacy:** Interfaces must be simple, mobile-responsive, and rely heavily on clear icons and minimal text input.
- **Zero Infrastructure Cost:** The solution must operate within free-tier cloud hosting (Vercel, Render/Railway, Supabase) and open-source stacks.
- **Non-Adjudicative Scope:** The application strictly organizes data and assists workflow; it does not issue legal approvals, guaranteed loans, or official government claim settlements.

5 Solution Approach

5.1 Structured prescription and dispensing

Analogous to clinical prescription vs. dispensing records, AgriAid maintains strict separation between financial inputs/claims documentation and final verified records:

- **Structured Farm Ledger Entries:** Financial data is recorded using standardized categories (Seeds, Fertilizer, Labor, Fuel, Machinery, Transport) with structured fields (Amount, Date, Crop Cycle, Yield Revenue) rather than unstructured notes.
- **Insurance Claim Record Separation:** Claim evidence uploaded by the farmer (photos, incident details) is maintained as an unalterable historical log, while status updates and missing item checklists are tracked separately.

5.2 Web architecture

AgriAid uses a 3-tier, service-oriented architecture:

- **Frontend:** React.js + Tailwind CSS providing responsive user dashboards and icon-driven interfaces.
- **Backend API:** Node.js + Express.js REST APIs handling authentication, business logic, scheduling conflict algorithms, and eligibility rule engines.
- **Database & Storage:** PostgreSQL for structured relational data (users, resources, bookings, ledger entries) and Supabase Storage for claim documents/photographs.

5.3 Authentication and authorization

- JWT-based session tokens for secure stateless authentication.
- Role-Based Access Control (RBAC) supporting three primary roles: *Farmer*, *Resource Provider*, and *Administrator*.
- Seamless support for dual status (e.g., an individual acting as both a Farmer requesting labor and a Resource Provider listing a tractor).

5.4 NFC/RFID identity integration

(Mapped to Location & Resource Identity Mapping in AgriAid)

- Resources and farms are tied to geographic coordinate identifiers mapped via OpenStreetMap and Leaflet.
- Manual pin-drop fallback is provided where GPS geolocation is unavailable or inaccurate.

5.5 Queue management

(Mapped to Resource Scheduling & Labor Matching in AgriAid)

- **Atomic Booking Transactions:** Conflict-detection logic flags and prevents double-booking of machinery for overlapping time windows.
- **Labor Request Queue:** Local short-term labor requests are published into a location-radius queue where local workers/groups can view and respond based on availability.

5.6 CI/CD and fast delivery enabler

- Automated builds, linting, and testing via GitHub Actions.
- Unit and integration testing using Jest, Supertest, and Postman.
- Automatic staging deployment to Vercel (Frontend) and Render/Railway (Backend API).

6 Evaluation Criterion

6.1 Primary evaluation metric

Search and Matching Query Response Time: The duration taken between initiating a resource/scheme search and receiving matched results.

- **Target:** < 2 seconds response time for 95% of requests under typical prototype load.
- **Measurement:** Automated API latency logging and database transaction timestamps.

6.2 Secondary evaluation metrics

- **Booking Conflict Prevention:** 100% detection and flagging of overlapping booking windows.
- **Ledger Calculation Accuracy:** Correct summation of seasonal net outcomes across expense/revenue entries.
- **Usability:** Successful task completion by a first-time user without prior training.
- **System Availability:** $\geq 95\%$ uptime during the project evaluation window.
- **Authorization Security:** Zero unauthorized access to administrative rule configurations or private financial ledgers.

6.3 Pilot validation plan

- Populate database with synthetic regional farmer profiles, equipment listings, and government schemes.
- Recruit representative student/evaluator test users to execute workflows (resource booking, breakdown logging, labor posting, expense recording, scheme matching).
- Compare system execution metrics against baseline manual operational scenarios.

7 Scalability

- **Theoretical Scalability:** Layered service-oriented architecture, database normalization up to 3NF, indexed PostgreSQL queries, paginated API responses, and stateless JWT authentication.
- **Operational Deployability:** Free-tier managed cloud infrastructure (Supabase PostgreSQL, Vercel, Render) with environment-based configuration.

8 Engine availability heuristic

The technology stack relies entirely on production-grade, open-source, and freely available technologies:

- **Frontend:** React.js, Tailwind CSS
- **Backend:** Node.js, Express.js
- **Database & Storage:** PostgreSQL, Supabase Storage
- **Maps/Location:** Leaflet, OpenStreetMap
- **Testing & CI/CD:** Jest, Supertest, Postman, Git/GitHub, Vercel, Render

9 Project scope and deliverables

9.1 Initial deliverable

- User profile management and JWT authentication.
- PostgreSQL schema and database migrations.

- Single-resource listing and simple booking request flow.
- Basic Farm Ledger entry (expenses/revenue logging).
- Staging environment deployment and CI build setup.

9.2 Subsequent deliverables

- Group booking with automatic overlap conflict detection.
- Farm equipment breakdown reporting and nearby service locator.
- Short-term local labor request and matching queue.
- Rule-based government scheme eligibility engine and application checklist.
- Insurance claim record creation and document/photo upload.
- Administrative dashboard for scheme rule management.

9.3 Stretch deliverables

- UI Localization / regional language translation via string externalization.
- Provider service history and rating system.

9.4 Explicitly out of scope

- Full-scale e-commerce agricultural input/produce marketplace.
- Direct replacement of government procurement systems or mandis.
- Actual approval, adjudication, or payout of scheme applications and insurance claims.
- Complete banking, loan, or payment gateway infrastructure.
- Advanced AI-based crop disease diagnosis or automated crop recommendation.

10 Risks and mitigations

11 Summary

AgriAid provides a practical, deployable web platform designed to solve operational coordination, financial tracking, and benefit assistance challenges faced by rural farmers.

To achieve this within a 12-week academic semester, the team selected an **Agile-Incremental Hybrid SDLC Model**. Core infrastructure (Auth/Database) is built as a foundation increment, followed by three sprint-style increments representing each umbrella module. This isolates development risk, ensures continuous integration, and guarantees a demonstrable working product at each project checkpoint.

ID	Risk Description	Cat.	L	I	Mitigation Strategy
R1	Scheme eligibility rules become outdated as policies change.	Data	4	4	Store rules in admin-editable DB tables, not hardcoded logic.
R2	Resource availability drifts from real-world status.	Oper.	3	3	Prompt providers to confirm availability; auto-expire stale listings.
R3	Inaccurate location calculation degrades match quality.	Tech	3	4	Validate coordinates on entry; geo-distance library; manual pin fallback.
R4	Group bookings create scheduling conflicts for same slot.	Tech	3	3	Server-side atomic booking transactions; clear conflict UI.
R5	Labor-matching logic becomes overly complex.	Tech	2	3	Implement simple rule-based matching first; defer complex heuristics.
R6	Sensitive claim documents exposed due to weak access control.	Sec.	2	5	Enforce strict RBAC + JWT on endpoints; use signed expiring URLs.
R7	Low digital literacy reduces adoption / correct usage.	UX	3	4	Minimal-text, icon-driven UI; user testing early in cycle.
R8	Synthetic demo data weakens demonstration value.	Oper.	2	2	Curate realistic seed datasets of local schemes early.
R9	Team schedule slips due to academic overlap.	Sched.	3	3	Weekly milestone check-ins; buffer week before final deadline.
R10	Scope creep beyond core 6 components.	Sched.	3	3	Enforce scope boundary; defer new ideas to future work.

Table 1: Risk Assessment Matrix (L = Likelihood 1–5, I = Impact 1–5)